PRÁCTICA 3

Visualización con Matplotlib y Seaborn — consolidado de Pandas, NumPy y Scikit-Learn

1. Presentación

Esta práctica cierra el recorrido iniciado en la Práctica 1 (exploración con Pandas y NumPy) y continuado en la Práctica 2 (transformación con Pandas, NumPy y Scikit-Learn). El dataset de trabajo sigue siendo ecommerce_raw.csv, pero ahora se incorpora una tercera pregunta, además de "¿qué contiene?" y "¿cómo lo transformo?": ¿cómo lo muestro para que se entienda?

Se incorporan Matplotlib y Seaborn para representar visualmente lo que hasta ahora se venía describiendo con números y tablas. La lógica es la misma que se vio en la teoría de condición habitacional: primero se cargan y preparan los datos, después se decide qué relación se quiere mostrar, y recién ahí se elige el gráfico adecuado. El gráfico no reemplaza el análisis: lo organiza para que se pueda leer mejor.

Como esta práctica funciona además como repaso integrador de cara al parcial, la actividad final no se limita a graficar: retoma explícitamente pasos de diagnóstico (Práctica 1) y de transformación (Práctica 2) antes de cada gráfico, para practicar el proceso completo de punta a punta sobre el mismo dataset.

2. Objetivos

Al finalizar, se deberá poder construir gráficos de barras, histogramas, boxplots, gráficos de dispersión (scatter), gráficos de torta y mapas de calor de correlación, tanto con Matplotlib como con Seaborn; combinar varios gráficos en una misma figura con subplots; personalizar título, etiquetas, leyenda y grilla; guardar gráficos como archivo de imagen; y elegir el tipo de gráfico adecuado según el tipo de variable y la pregunta que se quiere responder.

El objetivo transversal es repasar el proceso completo de un proyecto de datos — cargar, observar, transformar, relacionar y graficar — y reconocer en qué etapa de ese proceso se ubica cada herramienta vista hasta ahora.

3. Antes de graficar: recordar el recorrido

Sobre ecommerce_raw.csv ya se recorrieron dos etapas. Conviene tenerlas presentes porque esta práctica las da por sabidas y las usa como punto de partida:

- Práctica 1 — Diagnóstico: dimensiones del dataset, tipos de datos, valores faltantes, duplicados, variables categóricas y estadística descriptiva, sin modificar nada.
- Práctica 2 — Transformación: normalización de formatos numéricos, tratamiento de nulos, eliminación de duplicados, corrección de valores fuera de rango, normalización de categorías, discretización (binning), encoding y rescalado, trabajando siempre sobre una copia (df_clean).
- Práctica 3 (esta práctica) — Visualización: representar gráficamente lo que ya fue diagnosticado y transformado, y agregar el análisis de relaciones entre variables mediante correlación.

Si graficamos antes de limpiar, el gráfico no sabe que esos datos tenían problemas — los muestra igual. Por eso esta práctica trabaja sobre df_clean (el resultado de la Práctica 2), no sobre el CSV original sin procesar.

4. Preparar el entorno

Continuar sobre el mismo notebook, o retomar df_clean desde donde quedó en la Práctica 2, e importar las bibliotecas de visualización:

```
import matplotlib.pyplot as plt
import seaborn as sns
```

Si no se conservó df_clean de la práctica anterior, se puede volver a generar repitiendo las transformaciones ya vistas, o partir directamente de df para los ejercicios que no las requieran.

5. Comparar categorías: gráfico de barras

El gráfico de barras es el más natural cuando se quiere comparar una magnitud entre categorías independientes (no hay una secuencia entre ellas). Cada categoría es una barra, y la altura permite comparar rápido.

```
df_clean['categoria'].value_counts().plot(
    kind='bar',
    title='Interacciones por categoría',
    xlabel='Categoría',
    ylabel='Cantidad',
    figsize=(9, 5),
)
plt.show()
```

Con Seaborn, el equivalente es countplot(), que cuenta las ocurrencias de cada categoría directamente a partir de los datos crudos, sin necesidad de calcular value_counts() antes:

```
sns.countplot(data=df_clean, x='dispositivo')
plt.title('Interacciones por dispositivo')
plt.show()
```

6. Comparar una medida entre categorías: barras agrupadas

Cuando en lugar de contar registros se quiere comparar el promedio de una variable numérica entre categorías (por ejemplo, el rating promedio de cada categoría de producto), primero se agrupa y se calcula el promedio, y luego se grafica el resultado.

```
rating_por_categoria =
df_clean.groupby('categoria')['rating'].mean().sort_values(ascending=False)
rating_por_categoria.plot(
    kind='bar',
    title='Rating promedio por categoría',
    ylabel='Rating promedio',
)
plt.show()
```

7. Distribución de una variable numérica: histograma

El histograma agrupa los valores numéricos en intervalos (bins) y muestra cuántas observaciones caen en cada uno. Permite ver dónde se concentran los datos, si la distribución es simétrica y si hay valores extremos.

```
plt.hist(df_clean['edad'], bins=20)
plt.title('Distribución de edad')
plt.xlabel('Edad')
plt.ylabel('Frecuencia')
plt.grid()
plt.show()
```

Con Seaborn, histplot() agrega además, de forma opcional, una curva de densidad estimada (kde=True) que suaviza la forma de la distribución:

```
sns.histplot(df_clean['precio_pagado'], bins=30, kde=True)
plt.title('Distribución de precio_pagado')
plt.show()
```

8. Distribución y outliers: boxplot

El boxplot (diagrama de caja) resume la distribución de una variable numérica en cuartiles y marca como puntos individuales los valores que se alejan demasiado del resto (outliers). Es especialmente útil para comparar la distribución de una variable entre varias categorías al mismo tiempo.

```
sns.boxplot(data=df_clean, x='categoria', y='precio_pagado')
plt.title('Precio pagado por categoría')
plt.xticks(rotation=45)
plt.show()
```

9. Relación entre dos variables: scatter

El gráfico de dispersión (scatter) ubica cada observación como un punto según los valores de dos variables numéricas. Es la herramienta principal para detectar visualmente si existe algún tipo de relación entre ellas, antes de calcularla numéricamente.

```
plt.scatter(df_clean['precio_base'], df_clean['precio_pagado'], alpha=0.3)
plt.title('Precio base vs. precio pagado')
plt.xlabel('Precio base')
plt.ylabel('Precio pagado')
plt.show()
```

Con Seaborn, scatterplot() permite además colorear los puntos según una tercera variable categórica (hue), lo que agrega una dimensión más a la lectura del gráfico sin necesidad de un gráfico 3D:

```
sns.scatterplot(
    data=df_clean,
    x='visitas_previas',
    y='compras_previas',
    hue='compra',
)
plt.title('Visitas previas vs. compras previas, según compra')
plt.show()
```

10. De la observación a la medida: correlación

Así como en la teoría de condición habitacional se calculó la correlación de Pearson entre indicadores, acá se aplica el mismo cálculo sobre las variables numéricas del dataset de e-commerce. El resultado es una matriz:

un valor entre -1 y 1 por cada par de variables, donde -1 indica relación lineal negativa, 0 indica poca relación lineal y 1 indica relación lineal positiva.

```
variables_numericas = ['edad', 'precio_base', 'descuento', 'precio_pagado',
                       'rating', 'visitas_previas', 'compras_previas', 'cantidad']
matriz_corr = df_clean[variables_numericas].corr(method='pearson')
```

11. De la matriz al mapa de calor

Leer una matriz de números resulta incómodo cuando hay varias variables. El heatmap de Seaborn representa cada coeficiente como un color, y con annot=True además muestra el valor numérico dentro de cada celda.

```
sns.heatmap(matriz_corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Correlación entre variables numéricas')
plt.show()
```

12. Correlación y causalidad

Una correlación alta muestra una relación estadística entre dos variables; no demuestra que una sea causa de la otra. La correlación puede ser positiva (ambas variables aumentan o disminuyen juntas) o negativa (cuando una aumenta, la otra disminuye).

Para poder hablar de causalidad no alcanza con la correlación: deben cumplirse además otras dos condiciones. La causa debe ocurrir antes que el efecto (prioridad temporal), y hay que poder descartar que la relación se deba a otra variable oculta que esté afectando a ambas al mismo tiempo (que la relación no sea espuria).

En este dataset, por ejemplo, encontrar una correlación entre visitas_previas y compras_previas es esperable y no requiere mayor justificación causal. Pero si apareciera una correlación entre dos variables sin relación lógica evidente, conviene sospechar de una variable oculta antes de sacar conclusiones apresuradas.

13. Proporciones: gráfico de torta

El gráfico de torta (pie) muestra cómo se distribuye un total entre categorías, en términos de proporción más que de magnitud absoluta. Conviene usarlo con pocas categorías (idealmente 2 a 5); con muchas categorías, un gráfico de barras suele comunicar mejor.

```
df_clean['compra'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    ylabel='',
    title='Proporción de interacciones con compra',
)
plt.show()
```

14. Varias vistas juntas: subplots

subplot() permite ubicar varios gráficos dentro de una misma figura, útil para comparar distintas representaciones de un mismo problema sin tener que mirar varias figuras separadas.

```
fig, ejes = plt.subplots(2, 2, figsize=(12, 8))

df_clean['categoria'].value_counts().plot(kind='bar', ax=ejes[0, 0],
title='Categorías')
ejes[0, 1].hist(df_clean['edad'], bins=20)
ejes[0, 1].set_title('Distribución de edad')
sns.boxplot(data=df_clean, x='categoria', y='precio_pagado', ax=ejes[1, 0])
ejes[1, 0].set_title('Precio pagado por categoría')
ejes[1, 0].tick_params(axis='x', rotation=45)
ejes[1, 1].scatter(df_clean['precio_base'], df_clean['precio_pagado'], alpha=0.3)
ejes[1, 1].set_title('Precio base vs. precio pagado')

plt.tight_layout()
plt.show()
```

15. Personalizar no es decorar

Cada gráfico necesita contexto para poder leerse sin ambigüedad: título (plt.title), qué representa cada eje (plt.xlabel, plt.ylabel), una grilla que ayude a ubicar los valores (plt.grid) y, cuando hay más de una serie, una leyenda (plt.legend) que identifique cada una. La personalización tiene sentido cuando ayuda a comunicar la información, no cuando decora sin motivo.

16. Guardar los gráficos

Guardar un gráfico como archivo de imagen no es un detalle menor: suele ser necesario incorporarlo a un informe, una presentación o enviarlo, y no siempre queda solo en el notebook.

```
plt.savefig('rating_promedio_por_categoria.png', dpi=150, bbox_inches='tight')
```

17. Matriz de selección de gráficos

La misma lógica de la teoría (tipo de variable → qué quiero representar → gráfico recomendado) aplicada al dataset de e-commerce:

Quiero comparar / mostrar	**Tipo de dato**	**Gráfico recomendado**	**Ejemplo en este dataset**
Comparar categorías	Categórica	Barras	Interacciones por categoria
Comparar una medida por categoría	Categórica + numérica	Barras agrupadas	Rating promedio por categoria
Ver distribución de una variable	Numérica continua	Histograma	Distribución de edad o precio_pagado
Ver distribución y outliers por grupo	Categórica + numérica	Boxplot	precio_pagado por categoria
Ver relación entre dos variables	Dos numéricas	Scatter	precio_base vs. precio_pagado
Cuantificar una relación lineal	Dos o más numéricas	Matriz de correlación	Variables numéricas del dataset
Ver proporciones de un total	Categórica (pocas categorías)	Torta	Proporción de compra vs. no compra
Comparar varias vistas a la vez	Cualquier combinación	Subplots	Panel con barras + histograma + boxplot + scatter

18. Actividad integradora (consolidado)

Esta actividad funciona como repaso integrador de las tres prácticas. Resuélvanla en un notebook que parta de ecommerce_raw.csv y reconstruya, en el mismo orden, el recorrido completo: diagnóstico → transformación → visualización. Cada ejercicio debe incluir el código y una breve interpretación cuando se solicite. Trabajar siempre sobre df_clean; el archivo RAW no debe modificarse.

1. Punto de partida
Carguen ecommerce_raw.csv, generen (o recuperen) df_clean con las transformaciones de la Práctica 2, y confirmen con df_clean.info() que las columnas clave ya son numéricas.

Código y respuesta:

2. Barras: categorías
Grafiquen con un gráfico de barras la cantidad de interacciones por categoria. Identifiquen la categoría con más y con menos interacciones.

Código y respuesta:

3. Barras: provincias
Grafiquen las 5 provincias con más interacciones. Comenten si esa distribución les resulta esperable o llama la atención.

Código y respuesta:

4. Barras agrupadas: rating por categoría
Calculen el rating promedio por categoria y grafíquenlo. Indiquen qué categoría tiene mejor y peor rating promedio.

Código y respuesta:

5. Histograma: edad
Grafiquen la distribución de edad con un histograma. Describan la forma de la distribución (simétrica, sesgada, con picos).

Código y respuesta:

6. Histograma: precio_pagado
Grafiquen la distribución de precio_pagado con Seaborn (histplot, con kde=True). Comparen con la distribución de precio_base.

Código y respuesta:

7. Boxplot: precio por categoría
Grafiquen un boxplot de precio_pagado agrupado por categoria. Señalen si alguna categoría muestra outliers evidentes.

Código y respuesta:

8. Boxplot: rating fuera de rango

Recuperando el hallazgo de la Práctica 2 (rating con valores fuera de 0-5), grafiquen un boxplot de rating antes y después de aplicar el tratamiento elegido en esa práctica. Comparen ambos gráficos.

Código y respuesta:

9. Scatter: precio base vs. precio pagado

Grafiquen la relación entre precio_base y precio_pagado. Expliquen por qué es esperable que exista una relación fuerte entre ambas variables.

Código y respuesta:

10. Scatter con hue: visitas y compras previas

Con Seaborn, grafiquen visitas_previas vs. compras_previas coloreando los puntos según compra. Interpreten si se distingue algún patrón entre quienes compraron y quienes no.

Código y respuesta:

11. Matriz de correlación

Calculen la matriz de correlación de Pearson entre al menos seis variables numéricas del dataset. Identifiquen el par con correlación más alta y el par con correlación más baja (en valor absoluto).

Código y respuesta:

12. Heatmap de correlación

Representen la matriz anterior con sns.heatmap(annot=True). Elijan una correlación del mapa e interpreten si tiene sentido conceptualmente.

Código y respuesta:

13. Correlación y causalidad

Para la correlación más alta que encontraron, evalúen las tres condiciones de causalidad vistas en la teoría (correlación, prioridad temporal, no espuria). Concluyan si alcanza la evidencia para hablar de causalidad o no.

Código y respuesta:

14. Torta: proporción de compra

Revisando primero que compra sea una variable binaria válida (Práctica 1, ejercicio 14), grafiquen con un gráfico de torta la proporción de interacciones con y sin compra.

Código y respuesta:

15. Countplot: dispositivo normalizado

Usando el dispositivo ya normalizado (Práctica 2), grafiquen con sns.countplot() la cantidad de interacciones por dispositivo.

Código y respuesta:

16. Subplots

Armen una figura de 2x2 que combine al menos cuatro de los gráficos anteriores. Justifiquen por qué eligieron mostrar esos cuatro juntos.

Código y respuesta:

17. Personalización

Tomen dos gráficos ya hechos y agréguenles título, etiquetas de ejes, grilla y leyenda (cuando corresponda). Comenten qué información quedaba ambigua antes de personalizarlos.

Código y respuesta:

18. Guardar gráficos

Guarden al menos dos gráficos como archivo de imagen con plt.savefig(). Verifiquen que se hayan generado los archivos.

Código y respuesta:

19. Matriz de selección propia

Completen su propia matriz de selección de gráficos con al menos seis filas, usando variables de este dataset que no hayan aparecido en los ejemplos de la práctica.

Código y respuesta:

20. Pregunta de negocio propia

Formulen una pregunta propia sobre el dataset (por ejemplo: ¿qué categoría tiene mayor tasa de conversión?, ¿el dispositivo influye en el precio pagado?). Respóndanla combinando al menos una transformación y un gráfico.

Código y respuesta:

21. Informe final integrador

Redacten entre 15 y 20 líneas repasando las tres prácticas: ¿cómo era el dataset originalmente?, ¿qué se transformó y por qué?, ¿qué relaciones se encontraron al graficar y correlacionar?, ¿qué preguntas quedan abiertas para una futura etapa de modelado?

Código y respuesta:

19. Preguntas de reflexión

- ¿Por qué conviene transformar los datos antes de graficarlos, y no al revés?
- ¿Qué diferencia hay entre un gráfico que describe y un gráfico que sugiere una conclusión que los datos no respaldan?
- ¿Por qué un gráfico de líneas podría ser engañoso para comparar provincias o categorías que no tienen un orden natural?
- ¿Qué significa que dos variables tengan correlación 0? ¿Implica que no hay ninguna relación entre ellas?
- ¿Por qué el mismo gráfico (por ejemplo, un histograma) puede construirse tanto con Matplotlib como con Seaborn? ¿Qué aporta cada biblioteca?
- ¿Qué riesgo tiene elegir un gráfico de torta cuando hay muchas categorías?
- ¿Por qué es importante guardar los gráficos y no solo dejarlos visibles en el notebook?
- ¿Qué relación hay entre la matriz de correlación y el heatmap? ¿Por qué no alcanza con mirar solo uno de los dos?

20. Qué cuidar en esta etapa

Todos los gráficos deben construirse sobre df_clean, no sobre el dataset original sin transformar. Cada gráfico debe tener título y etiquetas de ejes, aunque sea exploratorio. Ninguna correlación encontrada debe presentarse como causalidad sin justificar las tres condiciones necesarias. Esta práctica no incluye entrenar ningún modelo: el resultado esperado es un conjunto de visualizaciones que comunican lo que ya se diagnosticó y transformó.

21. Resultado esperado

El resultado es un notebook que retoma el dataset diagnosticado en la Práctica 1 y transformado en la Práctica 2, y lo representa visualmente con Matplotlib y Seaborn: distribución de variables, comparación entre categorías, relaciones entre variables numéricas y su correlación, eligiendo en cada caso el tipo de gráfico adecuado y justificando esa elección.

Con las tres prácticas completas, el dataset queda diagnosticado, transformado y visualizado: la etapa siguiente sería seleccionar variables y entrenar un primer modelo de Machine Learning.

22. Referencias y reseñas recomendadas

- McKinney, Wes. Python for Data Analysis. O'Reilly Media. Referencia central para Pandas, NumPy, exploración, limpieza y manipulación de datos.
- VanderPlas, Jake. Python Data Science Handbook. O'Reilly Media. Presenta de forma integrada NumPy, Pandas, Matplotlib y herramientas de ciencia de datos.
- Hunter, J. D. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 2007. Referencia académica sobre Matplotlib.
- Waskom, M. L. seaborn: statistical data visualization. Journal of Open Source Software, 2021. Referencia académica sobre Seaborn.
- Documentación oficial de Pandas, Matplotlib y Seaborn. Se recomienda utilizarla como referencia técnica para verificar el comportamiento exacto de cada función.